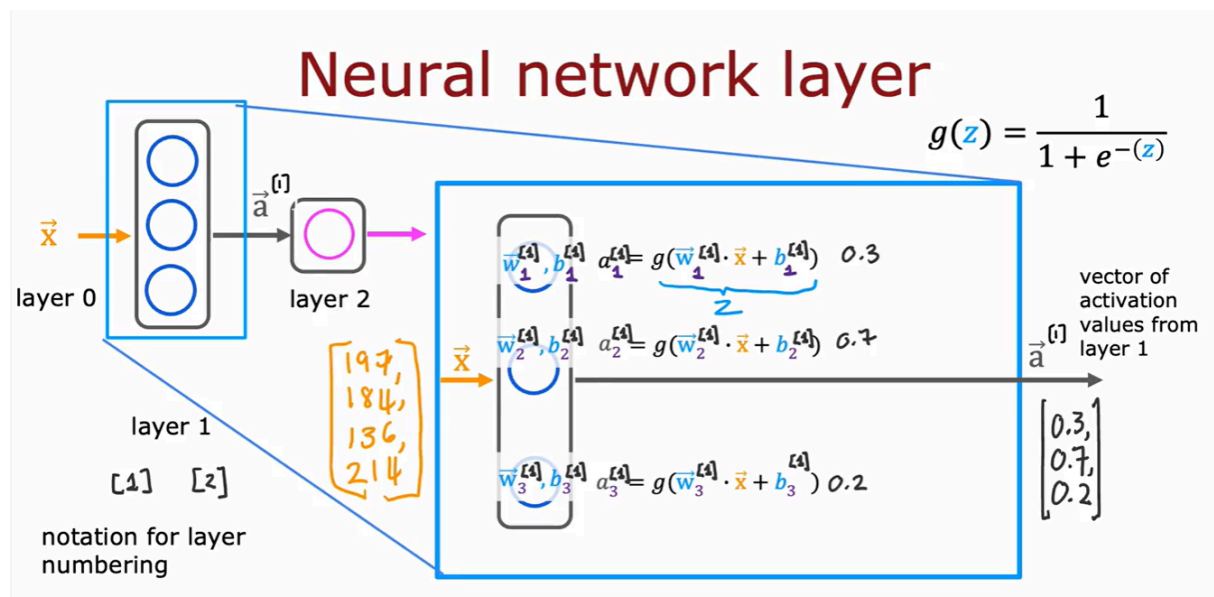
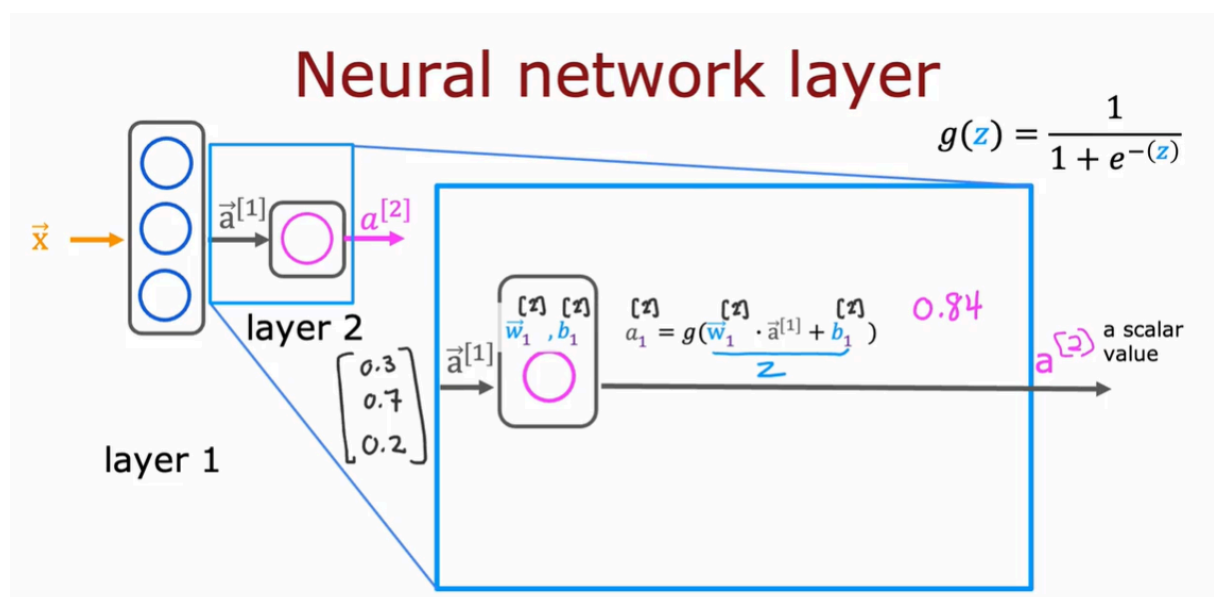


Neural network model

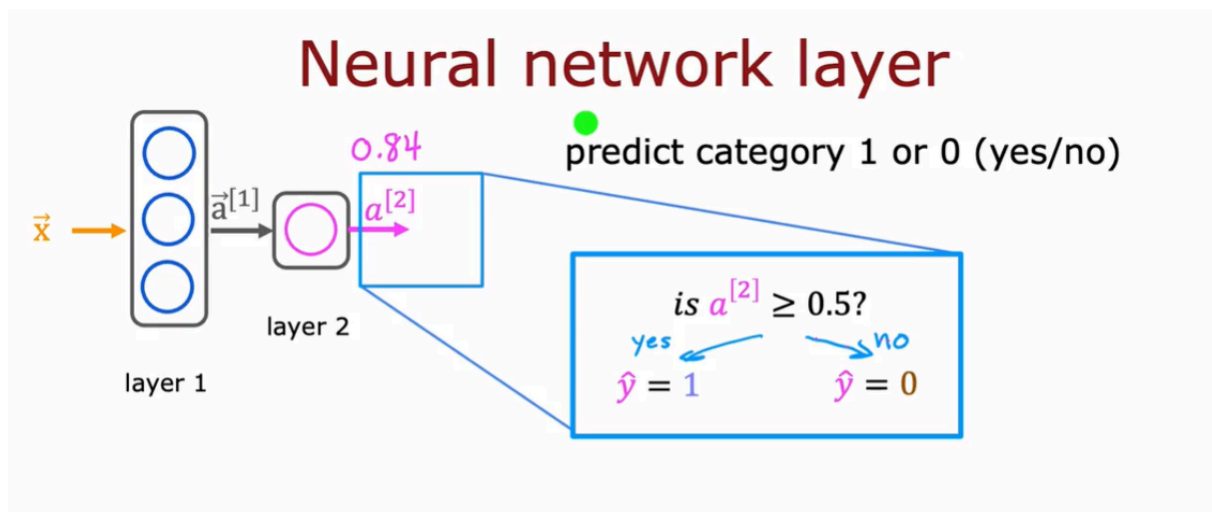


whenever you see this [1], that just refers to a quantity that is associated with layer 1 of the neural network. If you see superscript square bracket 2, that refers to a quantity associated with layer 2 of the neural network and similarly for other layers as well, including layer 3, layer 4 and so on for neural networks with more layers.

Now let's zoom into the computation of layer 2 of this neural network, which is also the output layer.



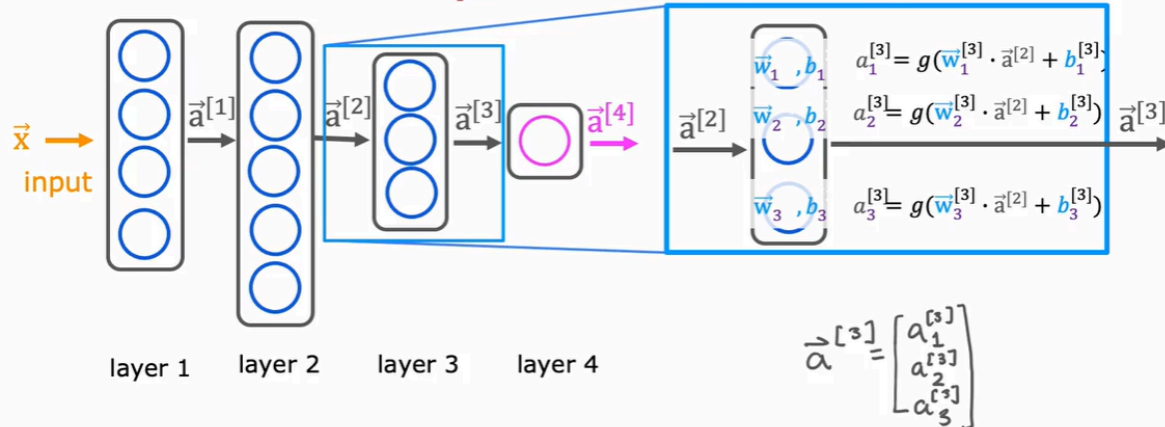
$a^{[2]}$ is a scalar because layer 2 only have 1 neuron giving 1 output. If there's more number of outputs, the $a^{[2]}$ should be vector like the output of layer 1



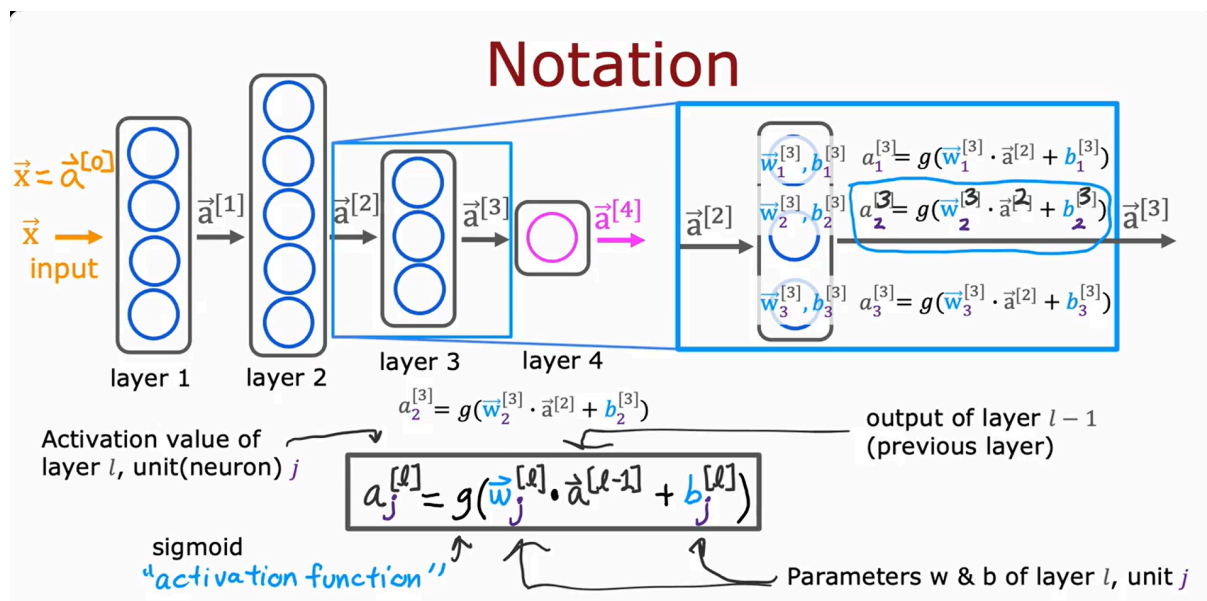
Once the neural network has computed $a^{[2]}$, there's one final optional step that you can choose to implement or not, which is if you want a binary prediction, 1 or 0, is this a top seller? Yes or no? As you can take the number $a^{[2]}$, and this is the number 0.84 that we computed, and threshold this at 0.5. If it's greater than 0.5, you can predict $\hat{y} = 1$ and if it is less than 0.5, then predict your $\hat{y} = 0$. We saw this thresholding as well when you learned about logistic regression in the first course of the specialization. If you wish, this then gives you the final prediction $\hat{y}$ as either one or zero, if you don't want just the probability of it being a top seller. So that's how a neural network works.

More complex neural networks

More complex neural network



We don't count the input layer, so this neural network has 4 layers. The output of layer 2 could be the input of layer 3, so that we have $a_1^{[3]} = g(\vec{w}_1^{[3]} \cdot \vec{a}^{[2]} + b_1^{[3]})$



- In the context of a neural network, g (the sigmoid function) has another name, which is also called the activation function, because g outputs this activation value.

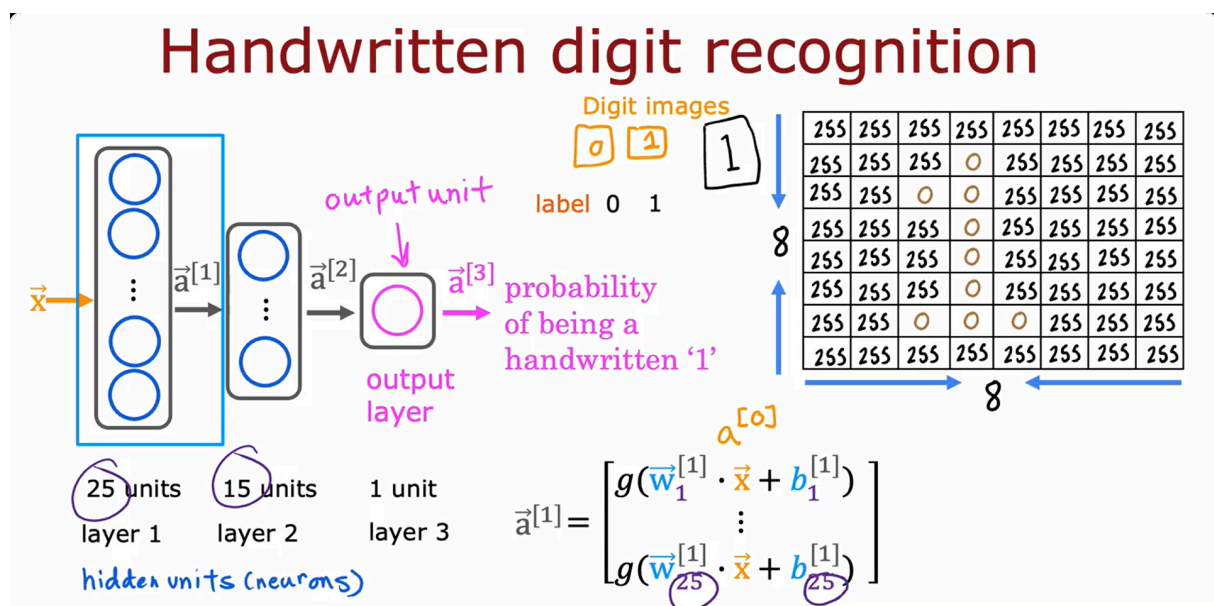
Activation value of layer l , unit(neuron) j

$$a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]})$$

- $a_j^{[l]}$: The activation value

- $g()$: The sigmoid function
- l : layer l
- j : unit(neuron)

Inference: making predictions (forward propagation)

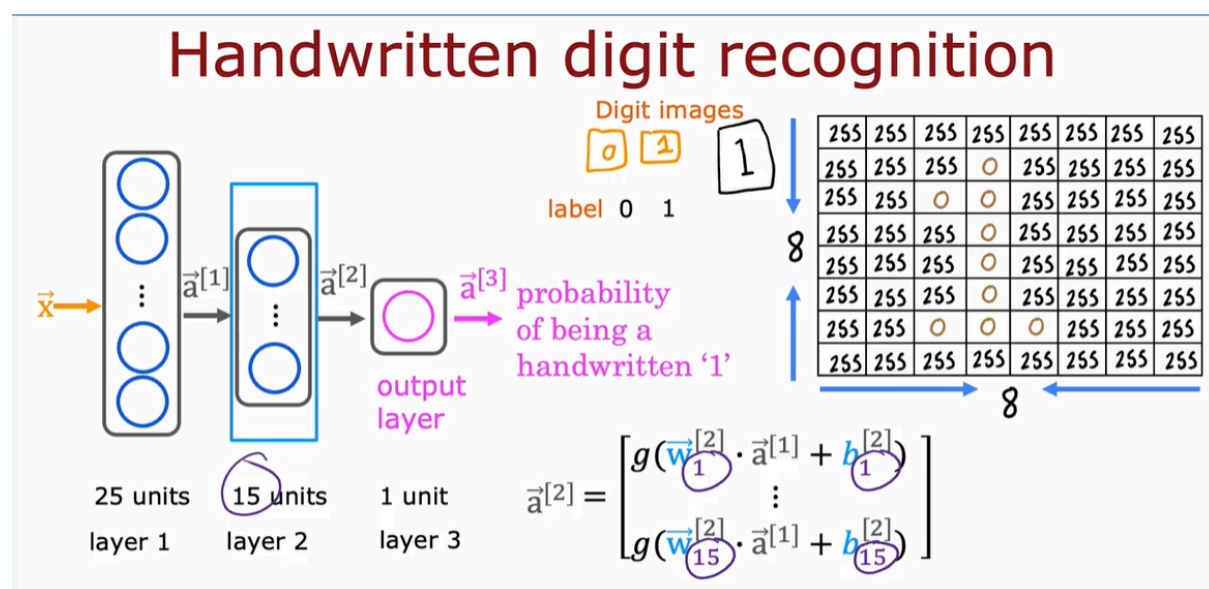


Let's take what we've learned and put it together into an algorithm to let your neural network make inferences or make predictions. This will be an algorithm called forward propagation. Let's take a look.

I'm going to use as a motivating example, handwritten digit recognition. And for simplicity we are just going to distinguish between the handwritten digits zero and one. So it's just a binary classification problem where we're going to input an image and classify, is this the digit zero or the digit one? And you get to play with this yourself later this week in the practice lab as well. For the example of the slide, I'm going to use an eight by eight image. And so this image of a one is this grid or matrix of eight by eight or 64 pixel intensity values where 255 denotes a bright white pixel and zero would denote a black pixel. And different numbers are different shades of gray in between the shades of black and white.

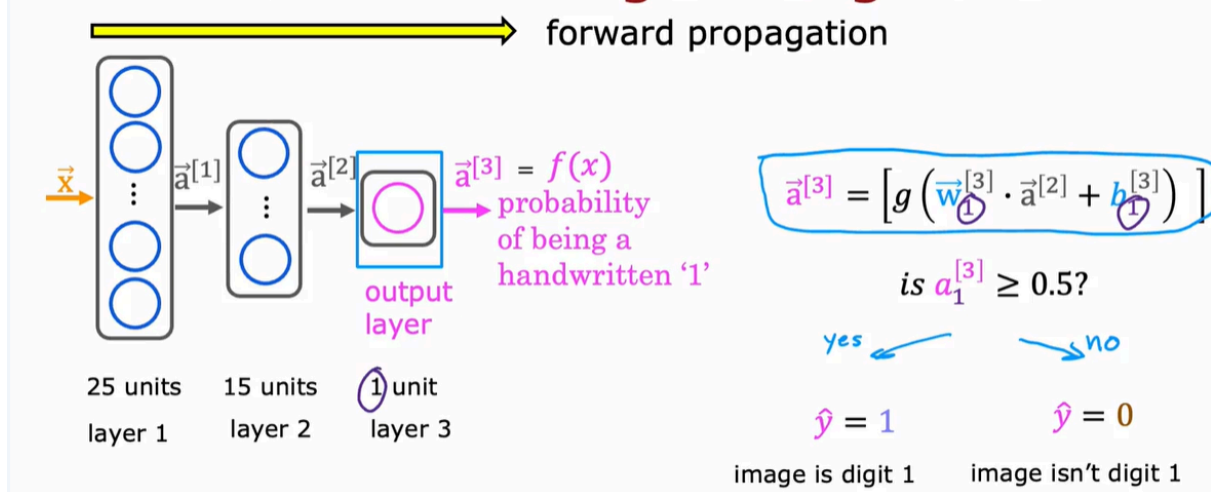
Given these 64 input features, we're going to use the neural network with two hidden layers. Where the first hidden layer has 25 neurons or 25 units.

Second hidden layer has 15 neurons or 15 units. And then finally the output layer or outputs unit, what's the chance of this being 1 versus 0?. So let's step through the sequence of computations that in your neural network will need to make to go from the input X , this eight by eight or 64 numbers to the predicted probability a_3 . The first computation is to go from X to a_1 , and that's what the first layer of the first hidden layer does. It carries out a computation of a super strip square bracket 1 equals this formula on the right. Notice that a_1 has 25 numbers because this hidden layer has 25 units. Which is why the parameters go from x_1 through x_{25} as well as b_1 through b_{25} . And I've written $\vec{x}$ here but I could also have written $a^{[0]}$ here because by convention the activation of layer zero, that is $a^{[0]}$ is equal to the input feature value $\vec{x}$. So let's just compute $\vec{a}^{[1]}$.



The image above is the output of layer 2.

Handwritten digit recognition



So the sequence of computations first takes $\vec{x}$ and then computes $\vec{a}^{[1]}$, and then computes $\vec{a}^{[2]}$, and then computes $\vec{a}^{[3]}$, which is also the output of the neural networks. You can also write that as $f(x)$.

So remember when we learned about linear regression and logistic regression, we use $f(x)$ to denote the output of linear regression or logistic regression. So we can also use $f(x)$ to denote the function computed by the neural network as a function of x . Because this computation goes from left to right, you start from $\vec{x}$ and compute $\vec{a}^{[1]}$, then $\vec{a}^{[2]}$, then $\vec{a}^{[3]}$.

- This algorithm is also called **forward propagation** because you're propagating the activations of the neurons. So you're making these computations in the forward direction from left to right.
- And this is in contrast to a different algorithm called backward propagation or back propagation, which is used for learning. And that's something you learn about next week. And by the way, this type of neural network architecture where you have more hidden units initially and then the number of hidden units decreases as you get closer to the output layer. There's also a pretty typical choice when choosing neural network architectures. And you see more examples of this in the practice lab as well. So that's neural network inference using the forward propagation algorithm. And with this, you'd be able to download the parameters of a neural network that someone else had trained and posted on the Internet. And you'd be able to carry out inference on your new data using their neural network